TECHNICAL REPORT

SharedPreferences Lab

Flutter Local Storage Application

In-depth code analysis, architecture review, and storage patterns guide

Project Name	shared_preferences_app
Version	1.0.0+1
Flutter SDK	Dart ^3.11.5
Package	shared_preferences ^2.5.5
Date	June 2, 2026
Report Type	Source Code Analysis & Documentation

1. Executive Summary

SharedPreferences Lab is a Flutter demonstration application built to showcase every core capability of the shared_preferences package (v2.5.5). The app implements a fully-featured settings screen that persists seven categories of user data across app restarts using device-local key-value storage.

The project was developed as a learning deliverable aligned with a structured curriculum covering Flutter local storage patterns. It demonstrates real-world usage of all five supported data types, covers both read and write paths, and includes cleanup operations (remove and clear). Bilingual UI support (English/Spanish) is achieved through persisted locale selection, further reinforcing SharedPreferences as a lightweight configuration store.

2. Project Overview

2.1 Purpose and Learning Objectives

The application was created to satisfy the following learning objectives:

- Understand what SharedPreferences is and when to use it over a full database
- Install and configure the shared_preferences Flutter package
- Use all five supported data types: String, int, bool, double, and List<String>
- Read stored values with safe null-coalescing defaults
- Remove individual keys and clear the entire preference store
- Implement practical use cases: theme toggle, language selection, notification toggle, login persistence, font scale, launch counter, and topic favourites
- Apply settings on cold app restart without any additional state management library

2.2 Technology Stack

Technology	Detail
Framework	Flutter (Material Design 3)
Language	Dart ^3.11.5
Storage Package	shared_preferences ^2.5.5
Theme Engine	Material3 with ColorScheme.fromSeed (Colors.teal), light + dark themes
State Management	Plain StatefulWidget + setState (no third-party library)

3. Application Architecture

3.1 Project Structure

The project follows a feature-first folder layout with a shared core layer. There are 12 Dart source files organised across three top-level directories:

```
lib/
  core/
    constants/  pref_keys.dart  |  app_strings.dart
    services/   preferences_service.dart
    theme/      app_theme.dart
  features/
    settings/
      models/   settings_state.dart
      screens/  settings_screen.dart
      widgets/  info_banner.dart
  shared/
    widgets/    hero_card.dart  |  hero_stat.dart
                section_card.dart  |  storage_badge.dart
  main.dart
```

3.2 Layer Responsibilities

Each layer has a distinct responsibility:

- core/constants — centralises all SharedPreferences key strings (PrefKeys) and all localised UI strings (AppStrings). This prevents magic-string bugs and makes key management refactor-safe.
- core/services — PreferencesService is a pure static helper with no Flutter widget dependency. It owns all SharedPreferences I/O, making it trivial to test in isolation and to swap storage backends in the future.
- core/theme — AppTheme generates ThemeData for both light and dark modes. The seed colour (Colors.teal) produces the full Material3 colour scheme automatically.
- features/settings/models — SettingsState is an immutable data class with a copyWith factory, representing the complete preference state at a point in time.
- features/settings/screens — SettingsScreen is the only screen; it loads state on init, drives all UI, and delegates every read/write to PreferencesService.
- shared/widgets — reusable display-only widgets (HeroCard, SectionCard, StorageBadge) that receive data through constructor parameters.

4. Data Model — SettingsState

SettingsState is an immutable Dart class that acts as the single source of truth for the UI and for all SharedPreferences writes. Every field maps directly to one preference key.

Key Constant	Dart Type	Default	Purpose / Notes

theme_mode	bool	false	Dark mode toggle. false = light theme.
notifications_enabled	bool	true	Push notification preference toggle.
remember_login	bool	true	Controls whether loggedIn and displayName are persisted.
logged_in	bool	false	Only written/read when rememberLogin is true. Removed on forgetLogin().
language_code	String	"en"	ISO 639-1 locale code. Drives full UI string translation.
display_name	String	"Guest"	Only written/read when rememberLogin is true. Removed on forgetLogin().
font_scale	double	1.0	MediaQuery textScaler multiplier. Range: 0.9 to 1.4.
launch_count	int	0	Incremented by +1 on every app load in PreferencesService.load().
favorite_topics	List<String>	["Flutter","SharedPreferences"]	Multi-select chip group. Written via setStringList.

The SettingsState.defaults() factory method provides safe fallback values used whenever a key is absent from storage (e.g., first launch). This eliminates null-safety issues at call sites.

5. PreferencesService — Storage Layer

5.1 Design Decisions

PreferencesService uses a private constructor (PreferencesService._()) to prevent instantiation; all methods are static. This is a deliberate choice for a single-class service that holds no instance state. The trade-off is reduced testability via dependency injection, which is acceptable for a learning project but would be refactored to an interface in production code.

5.2 The load() Method

load() is called once during SettingsScreen.initState(). It:

1. Acquires the singleton SharedPreferences instance via SharedPreferences.getInstance().
2. Reads every key with a null-coalescing default from SettingsState.defaults().
3. Increments the launch counter and immediately writes it back before returning.
4. Conditionally reads loggedIn and displayName only when rememberLogin is true, otherwise defaults to logged-out/Guest without touching storage.
5. Filters the favoriteTopics list against the known topic whitelist to discard any stale or corrupted entries.
6. Returns an immutable SettingsState with all resolved values.

The conditional login read in step 4 demonstrates a pattern important for privacy-sensitive apps: credentials are only loaded from disk when the user has explicitly opted in to persistence.

5.3 The save() Method

save() is called every time the user changes a preference (immediately, not on an explicit Save button). It writes all keys unconditionally except for loggedIn and displayName: these are either written (if rememberLogin is true) or actively removed (if false). This ensures that disabling "Remember login" immediately erases the stored credential keys rather than leaving stale data.

5.4 Specialised Operations

- forgetLogin() — removes only the loggedIn and displayName keys, used by the "Forget saved login" button.
- clearAll() — calls preferences.clear(), wiping every key in the application's namespace. The UI then resets to SettingsState.defaults() without a reload.

---

6. SharedPreferences API Patterns

6.1 All Five Write Methods

The app demonstrates every write method the package exposes:

Method	Used For	Code Site
setBool()	Theme, notifications, login flags	`preferences_service.dart, save()`
setString()	Language code, display name	`preferences_service.dart, save()`
setDouble()	Font scale (0.9 - 1.4)	`preferences_service.dart, save()`
setInt()	Launch counter	`preferences_service.dart, load() + save()`
setStringList()	Favorite topics list	`preferences_service.dart, save()`
remove()	Erase login keys selectively	`preferences_service.dart, forgetLogin() + save()`
clear()	Reset all app data	`preferences_service.dart, clearAll()`

6.2 Safe Read Pattern

Every read uses the null-coalescing operator (??) against a default:

```
final bool isDark = preferences.getBool(PrefKeys.themeMode) ?? defaults.isDarkMode;
```

This approach is superior to checking for null explicitly because it consolidates defaults in one place (SettingsState.defaults()), making them easy to find and change.

---

7. User Interface

7.1 Screen Structure

The app is a single-screen design (SettingsScreen) with a gradient background (primary container to surface). Content is a vertically scrollable Column containing:

7. HeroCard — gradient banner showing launch count, current user, and login status.
8. Preferences SectionCard — dark mode switch, notifications switch, remember-login switch, language dropdown, and font scale slider.
9. Login State SectionCard — display name text field, Save Profile button, Log In / Log Out button, and Forget Saved Login button.
10. Favourite Topics SectionCard — FilterChip grid for multi-select topic persistence.
11. Storage Patterns SectionCard — informational badges listing every API method used.
12. Clear All Data button — tonal filled button triggering a full preference wipe.

7.2 Localisation

The UI is fully internationalised via AppStrings.localized, a static map of language code to string key/value pairs. All display text passes through the _text(key) helper, which resolves against the currently persisted language_code. Switching the language dropdown immediately rebuilds the UI with the new locale because _persist() calls setState() before the async write completes.

Two locales are supported: English (en) and Spanish (es). Adding a third language requires only adding a new key to AppStrings.localized and a new DropdownMenuItem — no structural changes are needed.

7.3 Font Scaling

The fontScale preference (0.9x to 1.4x, five steps) is applied at the MaterialApp builder level using MediaQuery.copyWith(textScaler: TextScaler.linear(fontScale)). This means the scale applies to all text across the app — not just the settings screen — demonstrating how a single preference can affect global UI behaviour.

---

8. Storage Patterns Reference Guide

8.1 When to Use SharedPreferences

SharedPreferences is appropriate for:

- Small, primitive configuration values (fewer than ~100 keys)
- Data that changes infrequently and does not need indexing or querying
- Non-sensitive settings (not passwords, tokens, or PII — use flutter_secure_storage for those)
- First-time launch flags, tutorial-complete booleans, and similar one-shot state

SharedPreferences is NOT appropriate for:

- Lists of complex objects (use sqflite, Hive, or Isar)
- Large data sets (>50KB) — performance degrades as the preference file grows
- Data that must be queried, sorted, or joined
- Secrets, tokens, or credentials (use flutter_secure_storage)

8.2 Key Naming Convention

This app centralises all keys in PrefKeys as static String constants. Best practices:

- Use snake_case: "theme_mode", not "themeMode" (platform storage layers are case-sensitive)
- Prefix with an app namespace for multi-module projects: "myapp_theme_mode"
- Never hard-code key strings at the call site; always reference the constant

8.3 The Immutable State Pattern

Combining SharedPreferences with an immutable state object (SettingsState + copyWith) provides several advantages:

- All preference values are always in sync — you cannot have a live UI value that differs from the pending write value
- Reverting to defaults is trivial: replace the state with SettingsState.defaults()
- Unit-testing the model does not require mocking any I/O layer

8.4 Write-on-Change vs. Write-on-Save

This app uses write-on-change: every toggle, dropdown selection, or chip tap triggers an immediate SharedPreferences write. This is appropriate for settings because:

- The user expects preferences to survive a crash or force-quit at any moment
- Each write is cheap — a single key update on a small file

Write-on-save (accumulate changes, write only on an explicit Save button) is better suited to forms where partial entries are invalid and the user should be able to cancel. The profile display name uses this pattern, requiring a tap of "Save profile" to commit.

8.5 Conditional Key Removal

The login persistence logic is the most nuanced pattern in the app. Rather than writing a "not logged in" value, the app removes the keys entirely when "Remember login" is disabled. The benefit is that toggling the setting off genuinely erases the data rather than hiding it behind a flag — important for privacy and security hygiene.

---

9. Code Quality Assessment

9.1 Strengths

- Separation of concerns — storage, theme, constants, and UI are cleanly separated into their own files
- No third-party state management — the app proves that SharedPreferences + setState is sufficient for simple preference UIs
- Immutable model — SettingsState.copyWith() prevents accidental mutation
- Safe null handling — every read site uses ?? defaults, eliminating null-check boilerplate
- Privacy-aware storage — conditional writes/removes for login data demonstrate good data minimisation
- Internationalisation — the two-locale implementation is extensible without code changes

- Immediate persistence — settings survive unexpected app termination at any point
- Flutter lints enabled — analysis_options.yaml enforces recommended lint rules

9.2 Areas for Improvement

- PreferencesService testability — static methods cannot be injected; extracting an IPreferencesService interface would enable mocking in widget tests
- Error handling — the load() and save() methods do not catch platform exceptions; in production, SharedPreferences writes can fail on low-storage devices
- main.dart bootstrapping — the app calls runApp(const SettingsScreen()), making SettingsScreen also the MaterialApp root. Separating the app widget from the screen widget is cleaner
- No loading error state — if PreferencesService.load() throws, the app shows an infinite spinner
- Single language pair — extending to more than two languages would benefit from an ARB/intl pipeline rather than a hardcoded map

---

10. Deliverables Checklist

The table below maps each assignment deliverable to its implementation status:

Deliverable	Status	Evidence
App with SharedPreferences	✓Complete	`shared_preferences ^2.5.5 in pubspec.yaml`
Settings screen	✓Complete	`settings_screen.dart`
Theme persistence (light/dark)	✓Complete	`PrefKeys.themeMode, AppTheme`
Login state persistence	✓Complete	`PrefKeys.loggedIn, rememberLogin conditional`
Multiple preference examples	✓Complete	`9 keys across 5 data types`
setString / getString	✓Complete	`languageCode, displayName`
setInt / getInt	✓Complete	`launchCount`
setBool / getBool	✓Complete	`themeMode, notifications, rememberLogin, loggedIn`
setDouble / getDouble	✓Complete	`fontScale`
setStringList / getStringList	✓Complete	`favoriteTopics`
remove(key)	✓Complete	`forgetLogin() removes loggedIn and displayName`
clear()	✓Complete	`clearAll() wipes all preferences`
Apply settings on app restart	✓Complete	`load() called in initState()`
Language selection	✓Complete	`AppStrings.localized with en + es`
Notification settings toggle	✓Complete	`PrefKeys.notificationsEnabled`

README / storage patterns guide	✓Complete	This report (Section 8)

11. Conclusion

SharedPreferences Lab is a thorough, well-structured demonstration of the shared_preferences package. It covers every API method, uses all five supported data types, and implements seven real-world use cases that mirror production app requirements. The codebase is clean, readable, and lint-compliant, making it an effective learning artefact.

The most instructive design choices are the immutable SettingsState model, the single-file PreferencesService, and the centralised PrefKeys constants — together they show how to keep SharedPreferences usage maintainable as the number of preferences grows. The conditional login persistence demonstrates a data-minimisation pattern that applies directly to production privacy requirements.

The app fully satisfies all curriculum deliverables and provides a solid foundation for extending toward more advanced storage solutions (Hive, Isar, sqflite) or more sophisticated state management (Riverpod, BLoC) in subsequent learning stages.

End of Report